

QuickDrop

Simple Food Delivery API Service

PROJECT PROPOSAL

Prepared by

Adnan Khalil

adnankhalil131@gmail.com

July 14, 2026

1. Project Proposal

1.1 Project Overview

QuickDrop is a simple food-delivery website inspired by applications such as Toters. The system allows customers to create accounts, browse restaurants, view menus, add food items to a shopping cart, place orders, and follow the status of their orders.

The project will focus on the essential functions of a delivery platform. It will not include advanced features such as live GPS tracking, online payment gateways, artificial intelligence recommendations, or complex delivery-driver assignment.

The backend will be implemented as a Laravel REST API, while a simple website interface can be created using React or basic HTML, CSS, and JavaScript. The project will use MySQL for storing data.

1.2 Problem Statement

Customers need a convenient way to browse nearby restaurants and order food online. Restaurants also need a simple system for displaying their menu items and receiving customer orders.

QuickDrop solves this problem by providing one platform where:

- Customers can browse restaurants and menus.
- Customers can place food orders.
- Restaurants can manage their menu items.
- Administrators can manage users, restaurants, and orders.
- Order statuses can be updated and viewed.

1.3 Project Objectives

The main objectives of the project are to:

- Build a RESTful API using Laravel.
- Allow customers to register and log in securely.
- Display available restaurants.
- Display menu categories and food items.
- Allow customers to add items to a shopping cart.
- Allow customers to place orders.
- Allow customers to view their previous orders.
- Allow restaurant managers or administrators to update order statuses.
- Provide an administrator dashboard for managing the system.
- Store all system information in a structured MySQL database.

1.4 User Roles

Customer

- Register an account.
- Log in and log out.
- Manage personal information.
- Add delivery addresses.
- Browse restaurants.

- View restaurant menus.
- Search for food items or restaurants.
- Add items to a cart.
- Change item quantities.
- Remove items from the cart.
- Place an order.
- View order details.
- View order history.
- Cancel an order before preparation begins.

Restaurant Manager

- Log in.
- View the restaurant profile.
- Add menu categories.
- Add menu items.
- Edit menu items.
- Remove menu items.
- Mark items as available or unavailable.
- View incoming orders.
- Accept or reject orders.
- Update order statuses.

Administrator

- Manage users.
- Manage restaurants.
- Manage restaurant managers.
- Manage menu categories and menu items.
- View all orders.
- Update or cancel orders.
- View basic system statistics.

1.5 Main Functional Requirements

Authentication

The API will allow users to:

- Register.
- Log in.
- Log out.
- View their profile.
- Update their profile.

Laravel Sanctum can be used for API authentication.

Restaurant Management

The system will allow:

- Administrators to create restaurants.
- Restaurant managers to edit their assigned restaurant.
- Customers to view active restaurants.
- Customers to search restaurants by name or category.

Each restaurant will contain information such as:

- Restaurant name.
- Description.
- Address.
- Phone number.
- Image.
- Opening time.
- Closing time.
- Delivery fee.
- Minimum order amount.
- Availability status.

Menu Management

Each restaurant will have menu categories such as:

- Burgers.
- Pizza.
- Sandwiches.
- Drinks.
- Desserts.

Each category will contain menu items. A menu item will include:

- Name.
- Description.
- Price.
- Image.
- Availability status.

Shopping Cart

A customer can:

- Create a cart.
- Add items from a restaurant.
- Update item quantities.
- Remove items.
- View the subtotal.
- View the delivery fee.
- View the total price.

To keep the project simple, one cart can contain items from only one restaurant.

Order Management

A customer can place an order using the items in the cart. An order will contain:

- Customer.
- Restaurant.
- Delivery address.
- Ordered items.
- Item quantities.
- Item prices.
- Subtotal.
- Delivery fee.
- Total price.
- Payment method.
- Order status.
- Customer notes.

Order Statuses

The system will use the following statuses:

- pending
- accepted
- preparing
- out_for_delivery
- delivered
- cancelled
- rejected

Address Management

A customer can save multiple addresses, such as:

- Home.
- Work.
- University.

An address will contain:

- Address label.
- City.
- Area.
- Street.
- Building.
- Floor.
- Additional directions.

Payment Method

For the first version, the project will use:

- Cash on delivery.

An online payment system can be added later.

1.6 Non-Functional Requirements

Security

- Passwords must be encrypted.
- Protected routes must require authentication.
- Users must only access actions allowed for their role.
- User input must be validated.
- SQL injection protection must be provided through Laravel Eloquent.

Performance

- API responses should be returned quickly.
- Database queries should use relationships and indexing correctly.
- Restaurant and order lists should use pagination.

Usability

- The interface should be simple and responsive.
- Users should easily browse restaurants and menu items.
- Validation and error messages should be understandable.

Maintainability

- The project should use controllers, models, migrations, form requests, resources, services, and middleware.
- The code should follow Laravel naming conventions.
- API routes should be organized clearly.

1.7 Proposed Technology Stack

Component	Technology
Backend	Laravel REST API
Frontend	React with Vite
Database	MySQL
Authentication	Laravel Sanctum
Styling	Bootstrap or Tailwind CSS
API Testing	Postman
Development Environment	Visual Studio Code
Version Control	Git and GitHub

1.8 API Modules

The API will be divided into the following modules:

- Authentication API
- User Profile API
- Address API

- Restaurant API
- Menu Category API
- Menu Item API
- Shopping Cart API
- Order API
- Restaurant Manager API
- Administrator API

1.9 Example API Endpoints

Authentication

```
POST /api/register
POST /api/login
POST /api/logout
GET  /api/profile
PUT  /api/profile
```

Restaurants

```
GET    /api/restaurants
GET    /api/restaurants/{restaurant}
POST   /api/admin/restaurants
PUT    /api/admin/restaurants/{restaurant}
DELETE /api/admin/restaurants/{restaurant}
```

Menu Items

```
GET    /api/restaurants/{restaurant}/menu-items
POST   /api/manager/menu-items
PUT    /api/manager/menu-items/{menuItem}
DELETE /api/manager/menu-items/{menuItem}
```

Cart

```
GET    /api/cart
POST   /api/cart/items
PUT    /api/cart/items/{cartItem}
DELETE /api/cart/items/{cartItem}
DELETE /api/cart
```

Orders

```
POST /api/orders
GET  /api/orders
GET  /api/orders/{order}
POST /api/orders/{order}/cancel
PUT  /api/manager/orders/{order}/status
```

1.10 Future Improvements

The following features may be added in future versions:

- Live driver tracking.
- Driver accounts.
- Online card payments.
- Coupons and promotional codes.

- Customer reviews.
- Restaurant ratings.
- Push notifications.
- Favorite restaurants.
- Real-time order updates.
- Multiple languages.
- Delivery-area calculations.